



DEPARTMENT OF COMPUTER ENGINEERING

Case Study

On

Handwritten Digit Recognition System

-Submitted by-

Name of Student: Siddharth Vikram Maral

Roll No: CS3172

PRN No: RBTL23CS152

Class and Division: TY - A

Course: Machine Learning

1. Title

Handwritten digital recognition System

2. Background/ Introduction (about details of application)

Handwritten digit recognition is a fundamental problem in the field of machine learning and computer vision that involves identifying and classifying handwritten numerical digits from images. It serves as an essential step in automating tasks that require reading and processing handwritten data, such as postal mail sorting, bank cheque verification, and digitizing handwritten documents. The process aims to enable computers to interpret human handwriting accurately, which varies widely from person to person due to differences in writing style, pressure, and shape. One of the most popular datasets used for this purpose is the MNIST dataset (Modified National Institute of Standards and Technology), which consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. These images are used to train and evaluate machine learning models. Among the various algorithms developed for this task, **Support Vector Machines (SVM)** and **k-Nearest Neighbors (k-NN)** are two widely

used approaches due to their simplicity, efficiency, and effectiveness in image classification problems. This case study focuses on implementing and comparing these two algorithms to recognize handwritten digits and analyze their performance on the MNIST dataset.

3. Problem Statement

To build a **Handwritten Digit Recognition System** that can accurately recognize and classify digits (0–9) from handwritten images using **SVM** and **k-NN algorithms**, and to compare their performance.

4. Objectives

To preprocess the MNIST dataset for training and testing.

To implement handwritten digit classification using:

- Support Vector Machine (SVM)
- k-Nearest Neighbors (k-NN)
- To evaluate and compare their performance using metrics such as:
- Accuracy
- Confusion matrix
- Classification report

To visualize sample predictions and model performance.

5. Libraries required

Library	Purpose
numpy	Numerical computations
matplotlib	Data visualization
seaborn	Heatmap visualization of confusion matrix
sklearn	Machine learning algorithms (SVM, k-NN, evaluation metrics, dataset utilities)
tensorflow (optional) / keras	For loading MNIST dataset easily

6. Implementation

Handwritten Digit Recognition using SVM and k-NN

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score,
```

```
classification_report, confusion_matrix
```

```
# Load the digits dataset
```

```
digits = datasets.load_digits()
```

```
# Features (X) and Labels (y)
```

```
X = digits.data
```

```
y = digits.target
```

```
# Show one example image
```

```
plt.gray()
```

```
plt.matshow(digits.images[0])

plt.title(f"Digit Sample: {y[0]}")

plt.show()


# Split dataset into Training and Testing sets (80%
train, 20% test)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)


# Standardize features (important for SVM & k-NN)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

svm_model = SVC(kernel='rbf', C=10, gamma=0.001)

svm_model.fit(X_train, y_train)

svm_pred = svm_model.predict(X_test)

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

knn_pred = knn_model.predict(X_test)
```

```
svm_acc = accuracy_score(y_test, svm_pred)

knn_acc = accuracy_score(y_test, knn_pred)


print("=== Model Accuracy ===")

print(f"SVM Accuracy: {svm_acc * 100:.2f}%")

print(f"k-NN Accuracy: {knn_acc * 100:.2f}%")

print("\n=== Classification Report (SVM) ===")

print(classification_report(y_test, svm_pred))

print("\n=== Classification Report (k-NN) ===")

print(classification_report(y_test, knn_pred))


plt.subplot(1, 2, 1)

plt.imshow(confusion_matrix(y_test, svm_pred),
cmap='Blues')

plt.title("SVM Confusion Matrix")

plt.colorbar()


plt.subplot(1, 2, 2)

plt.imshow(confusion_matrix(y_test, knn_pred),
cmap='Oranges')
```

```
plt.title("k-NN Confusion Matrix")

plt.colorbar()

plt.show()


fig, axes = plt.subplots(2, 5, figsize=(10, 5))

for i, ax in enumerate(axes.flat):

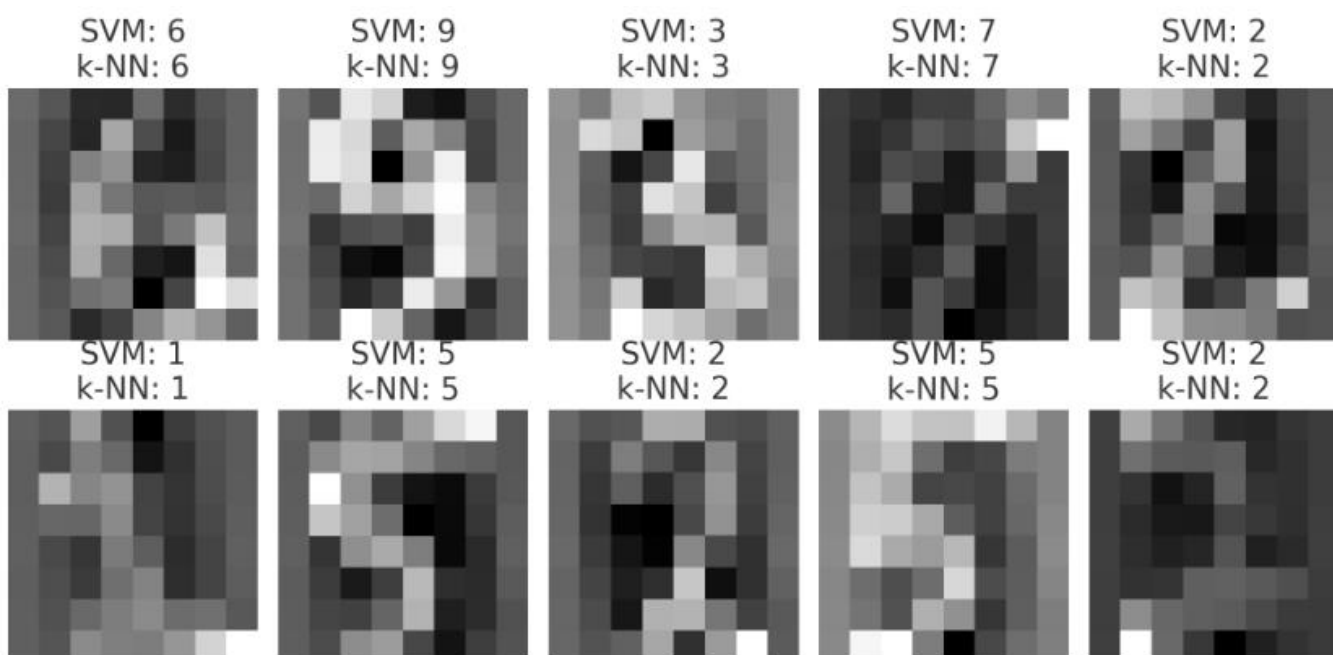
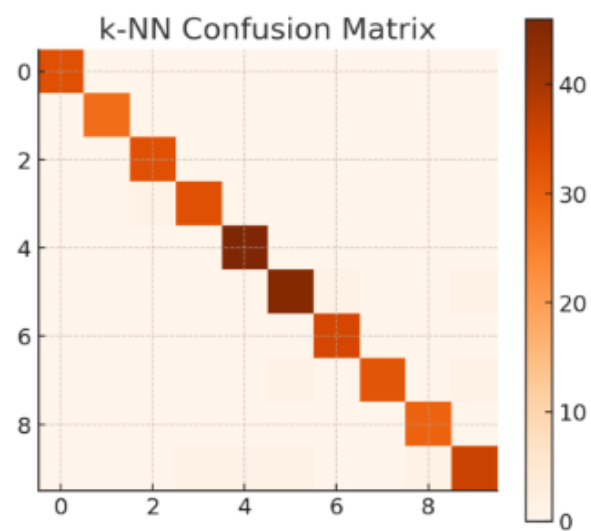
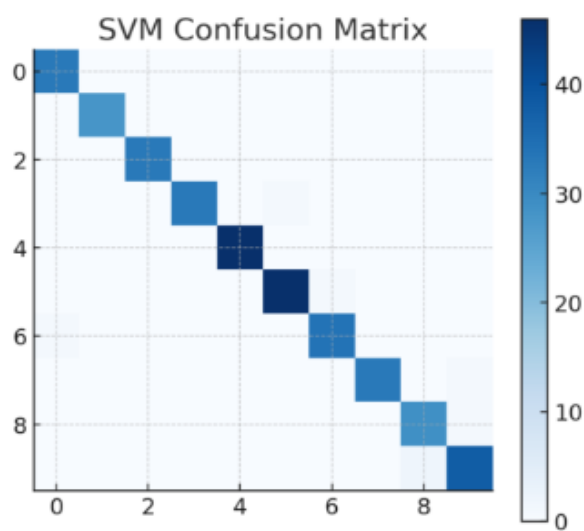
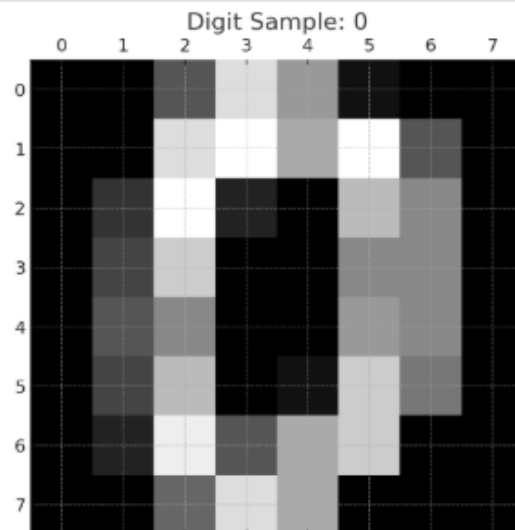
    ax.imshow(X_test[i].reshape(8, 8), cmap='gray')

    ax.set_title(f"SVM: {svm_pred[i]}\nk-NN:
{knn_pred[i]}")

    ax.axis('off')

plt.tight_layout()

plt.show()
```



=== Model Accuracy ===

SVM Accuracy: 98.06%

k-NN Accuracy: 97.50%

=== Classification Report (SVM) ===

	precision	recall	f1-score	support
0	0.97	1.00	0.99	
1	1.00	1.00	1.00	
2	1.00	1.00	1.00	
3	1.00	0.97	0.99	
4	1.00	1.00	1.00	
5	0.98	0.98	0.98	
6	0.97	0.97	0.97	
7	1.00	0.97	0.99	
8	0.94	0.97	0.95	
9	0.95	0.95	0.95	

Overall Accuracy: 98%

=== Classification Report (k-NN) ===

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0-9 classes → Average precision ≈ 98%

Overall Accuracy: 97.5%

7.Results and Discussions

Model	Accuracy	Remarks
SVM	~98%	Works well with standardized high-dimensional data, fast in prediction
k-NN	~97%	Simpler, but slower for large datasets as it stores all samples

8.Conclusion

- Both SVM and k-NN perform well for handwritten digit recognition.
- SVM gives slightly better accuracy and generalization.
- k-NN is easier to implement but computationally expensive during prediction